

Yelp Ratings Prediction Through A Text Only Approach

Shyam Patel

August 2025

Abstract

Yelp has long been a popular platform for users to search for businesses that cater to their preferences and screen them through peer reviews before visiting themselves. This study attempted to generate a meaningful yet relatively lightweight regressor of user rating based on text reviews associated with both a given user and business, without using collaborative filtering. The broader architecture used was a two-tower model, with a variety of encoding models and prediction head architectures examined. We predict star ratings for user/business pairs without the focal review text, using only other reviews. Results from the best model indicate a moderate improvement (RMSE of 0.876) over a mean baseline predictor (RMSE of 0.964), highlighting the challenging nature of this problem. The matrix factorization target on the same dataset achieved an RMSE of 0.856. The findings of this study have implications for both platforms like Yelp as well as consumers that would benefit from improved recommendation quality.

Keywords: Recommender systems, Two-tower models

Introduction

The two broad categories around recommendation models are collaborative filtering and content-based filtering. Collaborative filtering uses overlap between ratings across entities to predict the results of unseen interactions. This would be similar to having a friend with a known set of preferences and using their rating of a business to predict your own rating. Content-based filtering uses intrinsic characteristics of entities to predict ratings. An actual scenario analogous to this would be knowing that one prefers a particular cuisine in restaurants, then selecting for establishments that provide it. Matrix factorization is considered to be the standard for collaborative filtering and is often the highest performance strategy in practice.

The goal of this project was to explore different two tower implementations that could consider user review text in one tower and a combination of business review text as well as Yelp category tags in the other tower. Recommendations and ratings prediction is a ubiquitous problem in the modern era, with virtually every consumer tech company interested in serving recommendations closely aligned with the preferences of their customers. Recommendation was traditionally done with lighter techniques such as matrix factorization, although the increased availability of data, particularly unstructured data, have resulted in adoption of neural networks for this task¹. It is worth noting that computational cost is considerably higher with neural networks, which requires some additional utility to justify. The desired additional utility in this project was to have a recommendation model that produces predictions with similar accuracy through a content-based filtering approach, which can be useful to cover the gap when a user has low overlap with other users in their current market.

This is an inherently challenging problem as reviews of businesses can be very imprecise relative to user sentiment. Two users assigning a rating of “4 stars out of 5” could mean very different things in terms of their own sentiment towards that business. A study from 2021 has found that restaurants are on average rated 0.7 stars higher on Google Maps than they are on Yelp. Google Maps tends to be skewed towards the high end with many 5 star reviews, while Yelp tends to have more detailed and critical reviews, with a bimodal distribution of 1 and 5 star reviews². This results in unique problem domains from business to business, even within the same sector.

Background

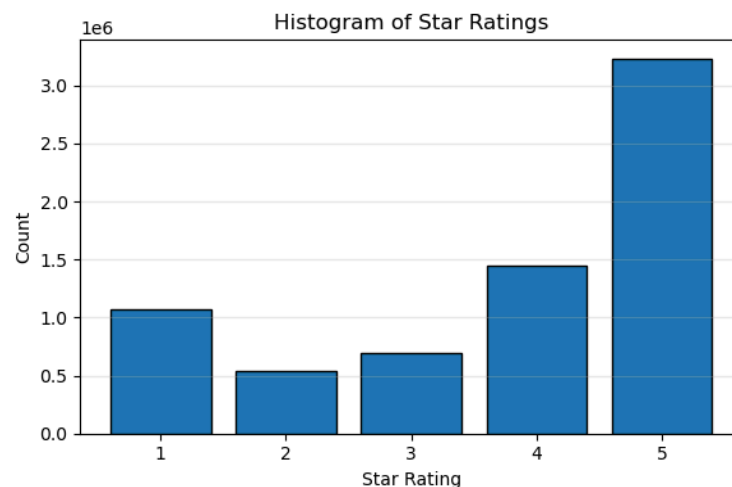
Likely due to commercial interest, much work has been done on recommendation systems around Yelp and other platforms. A study was done in 2025 that implements a very similar structure to what this study aims to examine. It implemented sentence level embeddings for user reviews and item attributes (businesses in the Yelp case), but with an extra layer of sophistication with CNN’s in each tower as well as an LSTM combining the two tower outputs. The order of improvement was approximately 10% which is considered to be large³.

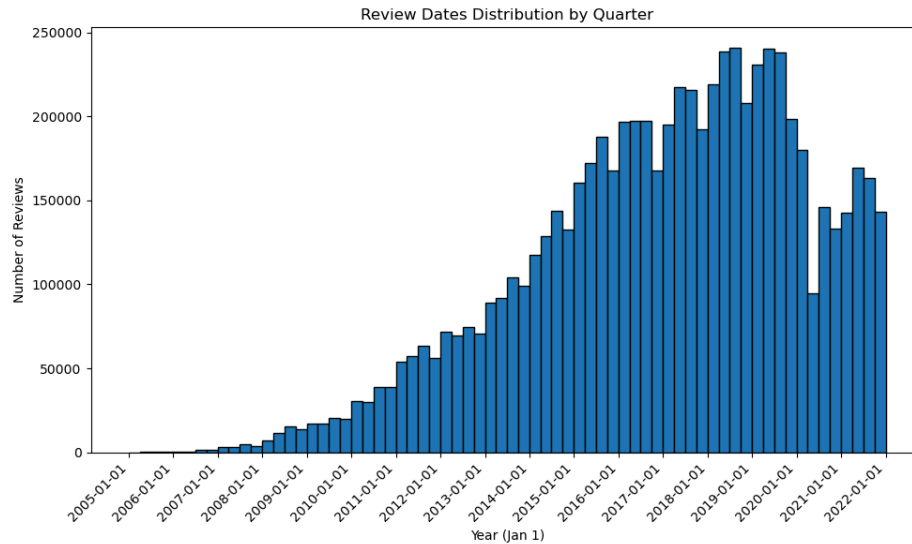
There has been an attempt to ignore time dependencies and use sentence level embeddings as well as item metadata in a two tower recommendation system for items on Amazon, with good results⁴. There has also been an attempt to use this sort of time independent architecture for business recommendations using short business descriptions as a classification task, which also yielded positive results⁵.

Methods

Data

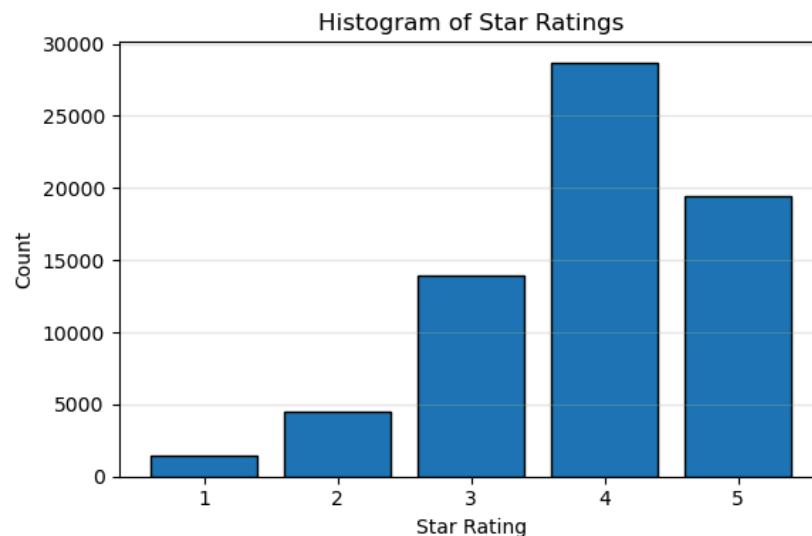
The data was sourced from the [Yelp Open Dataset](#). The data is anonymized on the user side and has many fields that include review text and business metadata. Initial exploration of the data revealed the following temporal spread of all reviews as well as the distribution of ratings.





Reviews had considerable temporal spread, from 2005 to 2022. While this was likely to result in significant time dependencies, the problem was framed without time dependencies to minimize the cold start problem as well as to control project scope. Duplicate reviews between a user and business were reduced to the most recent review to take the least disruptive reduction of resolution while simplifying data splits and validation. As expected, star ratings lined up with the bimodal distribution described earlier in the Introduction. This inherently posed some challenges as 2 and 3 stars reviews would be inherently neglected in the training process.

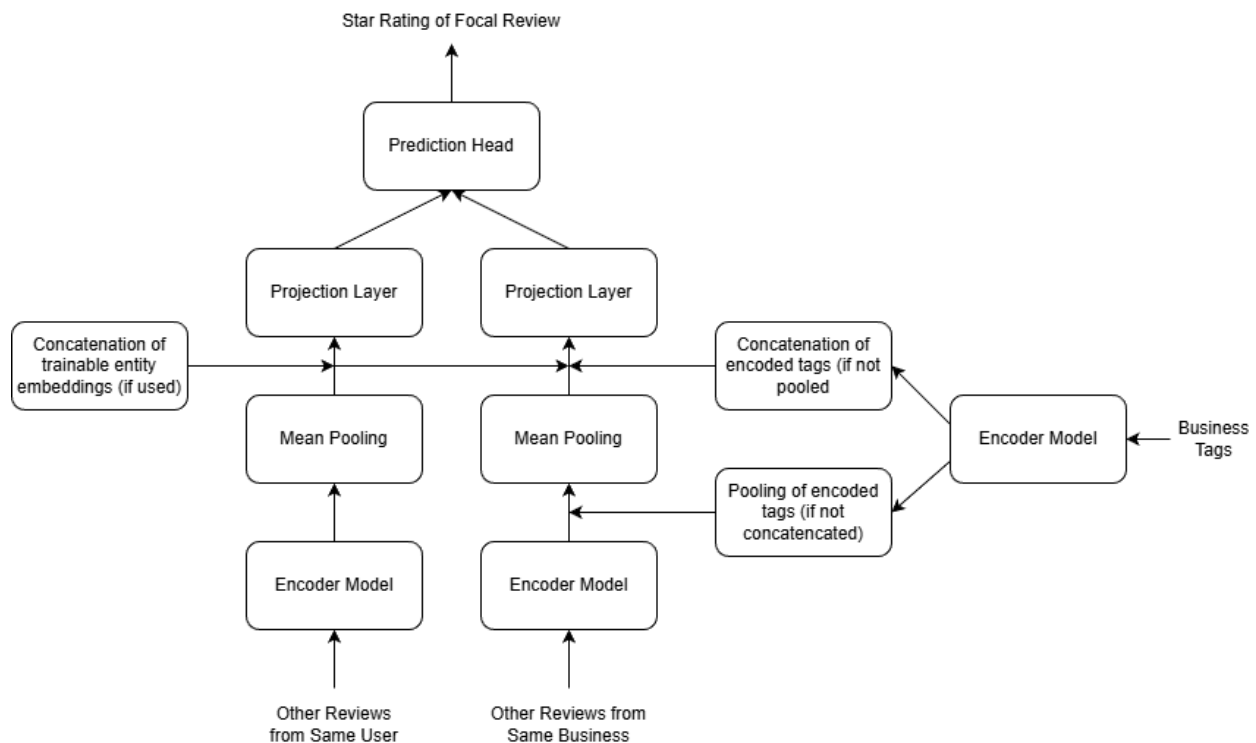
To maintain some level of efficacy in response to the lost time relationships, users with less than 60 reviews and businesses with less than 25 reviews were filtered out. I also required that available review datasets to construct corpuses on each tower be identical so that evaluation and data splits would be clear and easy to comprehend during experimentation. Most reviews ended up being clustered around the Philadelphia area. After the filtering, the ratings distribution was as follows.



This likely made the prediction task more difficult as ratings were now mostly concentrated on three adjacent values (3-5). The distribution before and after filtering both make classification a serious challenge with the class imbalance. It was also found that ~5% of reviews in the filtered set were over 512 tokens with a BERT tokenizer, meaning that they would be truncated in that model.

Features, Target, and Data Splits

The chosen feature to represent the user was the collection of all reviews for that user. The business was represented with all of the reviews for that business, along with the Yelp category tags (eg. Italian, Mexican, Japanese, etc). To generate training examples, the corresponding user and business was determined from each review. For each example, we exclude the focal review from both towers and construct pooled representations only from the same data split to prevent leakage. The star rating was extracted from the review as the target variable. While the loss function for training would be MSE, errors would be reported as RMSE to provide a more interpretable value.



Architecture

Each review on the user and business side of a training example was run through a specific encoder, after tokenizing appropriately. These review embeddings were then mean pooled to get a single embedding to represent that entity. With the focal review excluded, leakage was avoided. All mean pooling was done only within the appropriate split to avoid leakage of reviews even if it was not direct. Business category tags were similarly embedded and either pooled as another review or concatenated to the pooled review embedding on the business tower, with that being one of the experimental variables. Trainable entity embeddings were concatenated to these mean pooled representations if the configuration included this feature.

A projection layer was placed above each tower that was set to match the original embedding size of the encoder. This was to avoid compression of semantic meaning while also allowing retraining of final tower embeddings toward the task at hand, despite preencoding each review. FFNN and dot product prediction heads were examined for each encoder and tag strategy. FFNN architecture was created to be relatively deep to allow for exploration despite the risk of eventual overfitting. Dot product prediction heads would allow the projection layer to train into embeddings that could be compared effectively with cosine similarity, assuming a convincing relationship is found. A diagram of the general architecture can be found above this section.

Baseline

The baseline of this regression task is the Word2Vec version of the embeddings with pooled category tags. For context, a trivial baseline of the mean star rating of the training split had an RMSE of 0.964 on the test set. A matrix factorization model run on the same datasets (an ideal target to reach that uses collaborative filtering) yielded an RMSE of 0.856.

Modeling

The following encoders were progressively examined during the course of experimentation: Word2Vec, 6 Layer SBERT, 12 Layer SBERT, and a 12 Layer JINA model. Some notable differences between them were the embedding dimensions with Word2Vec outputting 300 dimensions, SBERT outputting 384, and JINA outputting 768. Context limits also varied, with Word2Vec being unlimited (manually calculated word average vector as final encoding), SBERT limited to 512 tokens, and JINA manually capped at 2048 tokens. Modeling was done with high epochs and low learning rates to allow the best possible results, particularly when a model configuration was in danger of overfitting.

Trainable entity embeddings were left out in the initial phase. Once the best configuration of model, prediction head, and business tag inclusion was identified, trainable entity embeddings were added in.

Results and Discussion

The lowest validation RMSE for each configuration was as follows. Refer to attached notebooks for detailed parameters around training.

Encoder/Params	FFNN, Pooled	FFNN, Concatenated	Dot Product, Pooled	Dot Product, Concatenated
Word2Vec	0.948 (baseline)	0.928	0.924	0.924
6 Layer SBERT	0.934	0.931	0.935	0.927
12 Layer SBERT	0.922	0.919	0.937	0.926
JINA	0.963	0.945	0.965	0.935

A consistent finding was that concatenating business category tags almost always reduces RMSE. This is likely because the model can decide how heavily to weigh those relative to reviews. The best RMSE overall came from 12 Layer SBERT embeddings with a relatively deep prediction head. The best dot product model came from an iteration of Word2Vec by a small margin. This is surprising since generally having the added context from transformer layers had a positive effect on RMSE. It could be possible that the dot product prediction head simply doesn't have the capacity to make use of the added context. As it stands, the dot product does not confer any benefits on models over the baseline.

Another surprise is that JINA with its extended context window and full-size embeddings actually performed the worst in all cases. A possibility here is that ALiBi positional encodings might be inferior to the fixed positional embeddings. It could also be possible that the extended context is not significant in practice with the first 512 tokens covering most of the relevant meaning of a given review.

With the highest performance configuration being the 12 Layer SBERT with concatenated tag embeddings and a FFNN prediction head (RMSE of 0.919), trainable entity embeddings were added. A simple 1 dimension embedding would be able to model user and business level biases and only reduced RMSE to 0.916 (validation set), while increasing embeddings up to 384 dimensions (the same as the SBERT output) reduced test RMSE to 0.876. The dot product prediction head on top of the same architecture yielded a test RMSE of 0.880.

This suggested that the entity level biases explained little of the variation in this dataset once review text was already being considered and that semantic interactions between review text and entity played a relatively large role (ie the term "good" could suggest different ratings depending on which user wrote it and which business it was describing).

Conclusion

Rating prediction is a very difficult problem according to the literature. A study in 2018 outlines a sophisticated prediction approach with NLP and user/business contexts as well as a baseline using matrix factorization. The matrix factorization RMSE goes as low as 0.8042 and the sophisticated multi-pronged NLP approach goes to 0.7441, but with the very large caveat that the focal review text is also being fed to the model⁶. As has been confirmed in many sources, user ratings are an inherently uncertain target to predict.

With the best performance from this project we were able to close the RMSE gap between our baseline and matrix factorization target on the same dataset by ~81%, but without examining focal review text. This is strictly with mean pooling and elimination of all time dependencies and no collaborative filtering. While these models are likely not a suitable replacement for matrix factorization, they do highlight how predictive past review text can be towards future interactions even while excluding collaborative filtering. This could be particularly useful in specific situations

with a large review corpus and low local overlap (eg a prolific reviewer that has moved or is traveling to a new city).

Since the dataset was concentrated around one metro (Philadelphia), it would be prudent to consider a small trainable embedding for each metro as well to account for differences in preference and language by location when scaling out. Another next step for this project would be to reapproach it as a time series problem and attempt a strategy such as recency weighting when pooling review embeddings, prioritizing the most salient reviews on each tower. And finally, the dot product prediction head performs very slightly worse than the FFNN head but is likely the preferable strategy in production since it allows easy integration with existing vector store and retrieval solutions. A typical vector search method such as ANN could be used to cut down candidates to a given subset with the dot product method and then those candidates could be reranked with the FFNN prediction head and associated features to allow for a more expressive prediction where applicable.

References

- [1] P. Covington, J. Adams, and E. Sargin, "Deep Neural Networks for YouTube Recommendations," in *Proc. 10th ACM Conf. Recommender Systems (RecSys)*, Boston, MA, USA, Sep. 2016, pp. 191–198. doi: 10.1145/2959100.2959190.
- [2] H. Li and B. Hecht, "3 Stars on Yelp, 4 Stars on Google Maps: A Cross-Platform Examination of Restaurant Ratings," *Proceedings of the ACM on Human-Computer Interaction*, vol. 4, CSCW3, Article 25, Jan. 2021, pp. 1–25. doi: 10.1145/3432953.
- [3] Y. Su, Y. Li, and Z. Zhang, "Two-Tower Structure Recommendation Method Fusing Multi-Source Data," *Electronics*, vol. 14, no. 5, Art. 1003, Mar. 2025. doi: 10.3390/electronics14051003.
- [4] Hou Y., Li J., Luan J. et al., "Bridging Language and Items for Retrieval and Recommendation" (BLaIR), *Preprint / ArXiv*, Mar. 2024. doi: (if available); introduces a series of pretrained sentence embedding models bridging natural language review text with item metadata for recommendation tasks
- [5] Da F., Kou G., Peng Y., "Deep Learning Based Dual Encoder Retrieval Model for Citation Recommendation," *Technological Forecasting and Social Change*, vol. 177, Article 121545, 2022. doi: 10.1016/j.techfore.2022.121545
- [6] B. Wang, S. Xiong, Y. Huang, and X. Li, "Review rating prediction based on user context and product context," *Applied Sciences*, vol. 8, no. 10, Art. 1849, Oct. 2018. doi: 10.3390/app8101849.